

Objects

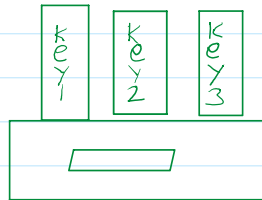
07 September 2026 19:00

As we know from the chapter Data Types, there are eight data types in Javascript. Seven of them are called primitive because their values contain only a single thing (be it a string or a number or whatever).

In contrast, objects are used to store keyed collection of various data and more complex entities. In Javascript, objects penetrate almost every aspect of the language. So we must understand them first before going in depth anywhere else.

An object can be created with curly braces {...} With an optional list of **properties**. A property is a "key:value" pair where **key** is a string (also called a property name) And a **value** can be anything.

We can imagine an object as a cabinet with signed files. Every piece of data is stored in its file by the key. It's easy to find a file by its name or add or remove a file.

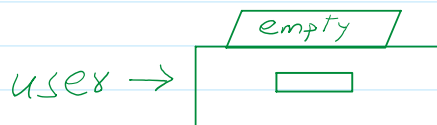


An empty object (empty cabinet) can be created using one of the two syntaxes:

```
let user = new Object(); // "object constructor" syntax
```

or

```
let user = {}; // "object literal" syntax
```



Usually, the curly braces {...} are used. That declaration is called an object literal.

Literal and properties

We can immediately put some properties into {... } as "key:value" pairs.

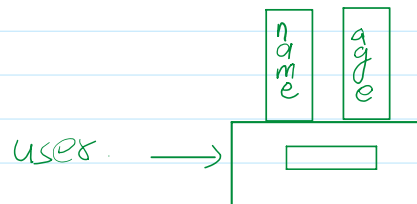
```
let user = {          // an object
  name: "John",       // by key "name" store value "John"
  age: 30              // by key "age" store value 30.
};
```

A property has a key (also known as "name" or "identifier") before the colon ":" and the value to the right of it.

In the user object there are two properties:

- The first property has the name "name" and the value "John".
- The second one has the name, "age" and the value 30.

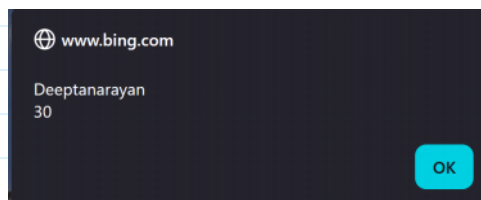
The resulting user object can be imagined as a cabinet with two signed file labeled *name* and *age*.



Property values are accessible using the dot notation.

```
>> let user = {  
  name: "Deeptanarayan",  
  age: 30  
};  
← undefined  
>> user  
← ▶ Object { name: "Deeptanarayan", age: 30 }
```

```
>> alert(user.name)  
← undefined  
>> alert(user.age)  
← undefined  
>> alert(user.name+"\n"+user.age)
```



The value can be of any type. Let's add a boolean one.

```
>> user.isAdmin = true;  
← true  
>> user  
← ▶ Object { name: "Deeptanarayan", age: 30, isAdmin: true }
```

==
- name.new property = value
object

```
>> user.isLoggedIn = "Yes";  
← "Yes"  
>> user  
← Object { name: "Deeptanarayan", age: 30, isAdmin: true, isLoggedIn: "Yes" }
```

To remove a property we can use the delete operator:

```
>> delete user.isLoggedIn;
< true

>> user
< Object { name: "Deeptanarayan", age: 30, isAdmin: true }
```

The last property in the list may end with a comma.

```
let user = {
  name: "John",
  age: 30,
};
```

That is called a trailing comma, Makes it easier to add, remove, move around properties because all lines become alike.

```
>> let student = {
  roll_num: 101,
  name: 'Jones',
}
< undefined

>> student
< ▶ Object { roll_num: 101, name: "Jones" }

>> |
```